

Neo4j

For fraud use cases in banking

With a live demo

Press **S** for speaker notes · **Esc** for overview · **F** for fullscreen

Agenda

1. **Introduction** — what is Neo4j?
2. **How is it different?**
3. **How is it better?**
4. **When is it better?**
5. **How do we get onboard?**
6. **Demo** — fraud, from questions to answers
7. **Conclusion**

Intro to Neo4j

What is Neo4j?

- The world's **most widely adopted graph database** (ranked no.1 db-engines/graphdb)
- Data stored as **nodes** and the **relationships** between them, not implied/computed like the relational model.
- Relationships are **first-class citizens**: stored, named, directed, and able to carry their own data.
- You query the *connections* directly with **Cypher** — a visual, pattern-based query language similar to SQL.

```
(Customer)-[:HAS_CARD]->(Card)  
      -[:FUNDS]->(Purchase)  
      -[:PAID_TO]->(Merchant)
```

Why it matters to the business

- Questions about *how things connect* become **simple and fast**.
- The model looks like the **whiteboard** the business already draws.

How is it different?

Graph Databases vs RDBMS

	Relational (RDBMS)	Graph (Neo4j)
Core unit	Tables & rows	Nodes & relationships/edges
Connections	Foreign keys, resolved at query time	Stored directly , traversed as pointers
"Who connects to whom?"	JOINS across tables	Follow the relationship
Cost of a deeper question	More JOINS, slower	Flat , per hop
Schema change	Migrations, ALTER TABLE	Additive , avoids downtime

- In an RDBMS, relationships are **inferred** manually by matching keys across tables.
- In a graph, relationships are **stored once** and simply **followed**, so connected queries stay fast as they get deeper.

Four joins vs one readable path

Find merchants paid by specific customer/card

SQL

```
SELECT DISTINCT m.name
FROM customer c
JOIN card cd ON cd.cif = c.cif
JOIN purchase p ON p.card_number = cd.card_number
JOIN merchant m ON m.name = p.merchant
WHERE c.cif = '5';
```

Cypher

```
MATCH (c:Customer {cif:'5'})-[:HAS_CARD]->(m)
      -[:FUNDS]->(:Purchase)-[:PAID_TO]->(m)
RETURN DISTINCT m.name;
```

Recursive queries like "anyone within 4 hops of a flagged account" is just `-[:SENT_TO|RECEIVED_BY*1..4]-`.

How is it better?

Filling a critical business gap

*The value in your data increasingly lives in the **connections**.*

- Real business questions are **relationship questions**: *fraud rings, recommendations, dependencies, supply chains, customer 360.*
- In a relational world those answers are **buried under join complexity** — slow to run and slow to build.
- Graph makes connected questions **cheap to ask and natural to express**, so the business can finally act on them.

Benefits

Unlock

- See patterns that span **multiple hops**
- Answers in at the **speed** of thought, not overnight batch
- Models that **match the business**

What you gain

- Detect **fraud rings**, not just bad transactions
- Discover **hidden clusters & cycles**
- Ship new questions **without re-architecting**

When is it better?

When to use graph databases?

If your hardest questions start with "how is X **connected** to Y **through...**", that's a graph problem.

1. **Primary data is highly networked**

Entities reference many others (customers ↔ accounts ↔ transfers ↔ merchants).

2. **And networked deeply**

Queries that span **many hops**, not just one join (rings, paths, reachability).

3. **Large transactional & analytical volumes exacerbate the problem**

At scale, JOIN-heavy queries slow down; graph traversal cost stays **flat per hop**.

4. **Relationship-gap in columnar data**

Warehouses/columnar stores aggregate beautifully but can't *traverse*; graph complements them where connections matter.

Where it pays off

Add graphs where connections drive **value**.

Strong fit

- Fraud & AML — rings, mules, shared identities
- Recommendations & personalization
- Knowledge graphs / GraphRAG for AI
- Network, IT & supply-chain dependencies
- Identity, entitlements, customer 360

Keep relational/columnar for

- Simple CRUD on isolated records
- Heavy tabular aggregation / reporting
- Flat data with few relationships

How do we get onboard?

Getting started

Start with the questions, not the schema. Reuse the data you already have:

1. **Start with the questions** you need to answer — or the places where **join-complexity** is hurting you today.
2. **Gather the important tables** that those questions touch (customers, cards, purchases, transfers...).
3. **Observe the columns** — keys become **relationships**, descriptive fields become **properties**.
4. **Design natively for the questions** — model the graph around how you'll *traverse* it, not around normalization.

```
Questions → Key tables → Columns → Native graph model
(or pain)  (the data)  → nodes,      (fits the questions)
                        rels, props
```

Demo — Fraud

What the demo showed

The banking fraud model, built the same way we just described:

1. **Domain modeled from the questions** — customers, cards, accounts, purchases, transfers and merchants, loaded directly from existing CSV exports.
2. **Questions powered the Cypher** — almost intuitively; the query *looks like* the question being asked.
3. **Graph algorithms run in the DB** — discover **cycles and clusters** that are effectively invisible in columnar/tabular form.

```
// Community detection groups colluding accounts into rings
CALL gds.louvain.stream('xfers')
YIELD nodeId, communityId;
```

Why the "old way" is hard

Relational / columnar

- Rules are limited per-transaction: "amount > X"
- Rings need **recursive self-joins** — slow, brittle, rarely built
- Clusters live across rows no single query connects
- Patterns surface **after the fact**, in batch

Graph (Neo4j)

- Traverse the **whole network** of money flow
- Cycles & communities are **built-in algorithms**
- Rings surface as **structure**, in real time
- New fraud patterns = **new query**, not new pipeline

Fraud rings operate as a *network*. Graph lets you investigate them as one.

Conclusion

Business value

- **Ask the hard questions** — connected, multi-hop questions become simple and fast.
- **Fast answers** whereas relational models needs overnight batch — e.g. catching fraud *as it happens*.
- **Faster delivery** — model matches the business, schema evolves additively.
- **Higher-value insight** — find **rings, cycles and clusters**, beyond isolated records.
- **Protects existing investment** — complements your RDBMS/warehouse; adopt incrementally where connections drive value.

Impact

- **Revenue & loss avoidance** — detect fraud rings and risk earlier; better recommendations lift conversion.
- **Speed to insight** — queries answered in real time, projects delivered in weeks.
- **Agility** — evolve the model as the business changes.
- **Competitive edge** — act on the connections in your data before competitors.

The connections in your data are an asset. Neo4j lets you use them.